

Práctica 3: Búsqueda con retroceso - Ubicación de centros de urgencias

Algoritmia Básica · Grado en Ingeniería Informática · Curso 2025/26

David Puértolas Merenciano (NIA: 900584) · Ibón Castarlenas Cortés (NIA: 900085)

1. Descripción del problema

Se considera un grafo no dirigido ponderado $G=(V,A)$, donde cada vértice representa una localidad y cada arista ponderada representa el tiempo de desplazamiento entre dos localidades. Dado un conjunto E incluido en V de centros ya existentes y un entero k , el objetivo es seleccionar un subconjunto S incluido en $(V - E)$, con $|S|=k$, que minimice el peor tiempo de acceso desde cualquier localidad al centro más cercano (existente o nuevo).

Formalmente, se minimiza:

$\max_{v \in V} \left[\min_{u \in (E \cup S)} d(v,u) \right]$ donde $d(v,u)$ es la distancia mínima en el grafo.

2. Diseño del algoritmo

2.1 Preprocesado de distancias

Como el criterio objetivo depende de distancias mínimas entre cualquier par de localidades, se aplica **Floyd-Warshall** sobre la matriz de adyacencia inicial. Tras este paso, $\text{dist}[i][j]$ contiene el tiempo mínimo real entre i y j .

2.2 Representación de soluciones y restricciones

Una solución se representa como una tupla ordenada $(s_1, \dots, s_k)$ con s_i perteneciente a $(V - E)$ y $s_1 < s_2 < \dots < s_k$.

La condición de orden estricto impone la restricción implícita que elimina permutaciones equivalentes: se exploran combinaciones, no arreglos.

2.3 Espacio de soluciones

Sea $N_c = |V - E| = n - c$ el número de candidatas. El árbol de búsqueda tiene profundidad k ; en cada nivel se elige una nueva localidad con índice creciente.

El número de hojas es: $C(N_c, k) = \text{"Nc sobre k"} = N_c! / (k! * (N_c - k)!)$.

Por tanto, el crecimiento combinatorio viene dado por $O(C(N_c, k))$.

2.4 Función de evaluación y poda

Para una solución parcial o completa se calcula, para cada localidad v , su distancia mínima al conjunto de centros considerado. El valor de la solución es el máximo de dichas distancias.

Se emplean dos mecanismos de reducción de búsqueda:

1. **Cota superior inicial voraz (greedy):** antes del retroceso se construye una solución factible rápida, colocándose cada nuevo centro donde más se reduce el peor acceso actual. Esta solución inicial fija una cota superior útil.
2. **Poda en nodos parciales:** durante el backtracking se calcula una cota parcial del peor acceso con los centros ya fijados. Si no existe posibilidad razonable de mejorar la mejor cota actual (según la localidad crítica y candidatos restantes), la rama se descarta.

3. Implementación

El programa se implementa en C99 (*code/ubicaCentros.c*) y se ejecuta como:

```
./ubicaCentros <entrada> <salida>
```

La salida por caso sigue el formato:

```
tiempo_ms n_nodos valor_optimo s1 s2 ... sk
```

donde **n_nodos** es el número de nodos del árbol generados y **s1..sk** aparece en orden creciente.

Estructura principal de la implementación:

- Lectura de caso y construcción de matriz de distancias.
- Floyd-Warshall.
- Cálculo de distancia de cada localidad al centro existente más cercano.
- Construcción del vector de candidatas.
- Cota inicial greedy.
- Backtracking combinatorio con poda.
- Escritura de resultados.

Para facilitar la repetición de pruebas se incluye *ejecutar.sh*, que compila, lanza verificaciones y ejecuta los experimentos. Además, *tools/generar_pruebas.py* genera instancias aleatorias conexas de forma reproducible (semilla fija por defecto).

4. Análisis de costes

El coste total del programa por caso puede descomponerse en:

1. **Floyd-Warshall:** $O(n^3)$.
2. **Búsqueda con retroceso:** en el peor caso, $O(C(n-c, k) * n * k)$, al evaluar nodos sobre todas las localidades y centros considerados.

En la práctica, la poda y la cota inicial reducen de forma apreciable el número de nodos realmente explorados, alejando el comportamiento real del peor caso teórico en los tamaños probados.

5. Verificación de correctitud

Se han validado los resultados en tres niveles:

1. **Ejemplos del enunciado**(*pruebas/ejemplo_enunciado.txt*):
 - Caso 1: valor óptimo 10, nuevo centro 5.
 - Caso 2: valor óptimo 3, nuevos centros 1 y 3.
2. **Casos triviales verificables a mano** (*pruebas/caso_trivial.txt*): valores óptimos 5, 3 y 1.
3. **Caso intermedio** (*pruebas/caso_mediano.txt*): valor óptimo 6, solución 2 y 5.

En todos los casos analizados, las salidas son coherentes con el comportamiento esperado del problema.

6. Experimentación y resultados

Se evalúan dos métricas:

- **tiempo_ms**: tiempo de ejecución por caso.
- **n_nodos**: nodos del árbol de búsqueda generados.

6.1 Experimento 1: variación de n con k=2

Archivo: *pruebas/exp_variar_n.txt*

Resultados: *resultados/resultado_variar_n.txt*

n	tiempo_ms	n_nodos
8	0.00	4
10	0.00	9
12	0.00	12
15	0.00	10
18	0.00	16
20	0.00	17
25	0.01	19

Se observa crecimiento suave de **n_nodos** y tiempos muy bajos en todo el rango.

6.2 Experimento 2: variación de k con n=15

Archivo: *pruebas/exp_variar_k.txt*

Resultados: *resultados/resultado_variar_k.txt*

k	tiempo_ms	n_nodos	valor_optimo
1	0.00	1	35
2	0.01	14	33
3	0.03	77	31
4	0.11	274	28
5	0.21	544	25
6	0.14	186	23

El valor óptimo mejora al aumentar k (disminuye el peor tiempo de acceso), mientras que el coste de búsqueda tiende a crecer por el aumento del espacio combinatorio. La no monotonicidad en **n_nodos** para k=6 se explica por el efecto de la poda.

6.3 Experimento 3: instancias grandes

Archivo: *pruebas/exp_grande.txt*

Resultados: *resultados/resultado_grande.txt*

Caso	Configuración	tiempo_ms	n_nodos
1	n=30, k=3	0.12	238
2	n=40, k=3	0.39	400
3	n=50, k=3	1.00	603

La tendencia confirma aumento de coste con el tamaño del problema, manteniendo tiempos prácticos reducidos para los rangos evaluados.

7. Conclusiones

La solución implementada resuelve **correctamente el problema de ubicación óptima** de centros bajo criterio minimax de tiempo de acceso. El diseño combina un preprocesado de caminos mínimos con una **búsqueda exacta por retroceso**, reforzada mediante cota inicial voraz y poda en nodos parciales.

Los experimentos muestran:

1. Correctitud en ejemplos oficiales y casos manuales.
2. Mejora sistemática de cobertura al incrementar k.
3. Número de nodos explorados claramente inferior al de una enumeración ciega en los tamaños ensayados.
4. Tiempos de ejecución bajos en todos los escenarios experimentales realizados.

En consecuencia, la implementación es adecuada para los objetivos de la práctica: exactitud en la solución, trazabilidad de métricas (*tiempo_ms*, *n_nodos*) y reproducibilidad de pruebas.